

ZYVOR · SHUKRA

See inside every VM, **without going inside it.**

A brochure for teams that run KVM: how Shukra watches every QEMU workload from the hypervisor with eBPF, what happens step by step when a VM turns slow or its traffic stops, and everything else it does.

OBSERVE

KVM exits, run-queue delay, disk latency and every TCP connect a guest makes. No agent in the VM.

EXPLAIN

Ranked host-side causes with the evidence, for now or for 03:12 last night.

PROTECT

Detection rules, learned baselines, and isolate a compromised VM without touching it.

SAY WHAT IT CANNOT SEE

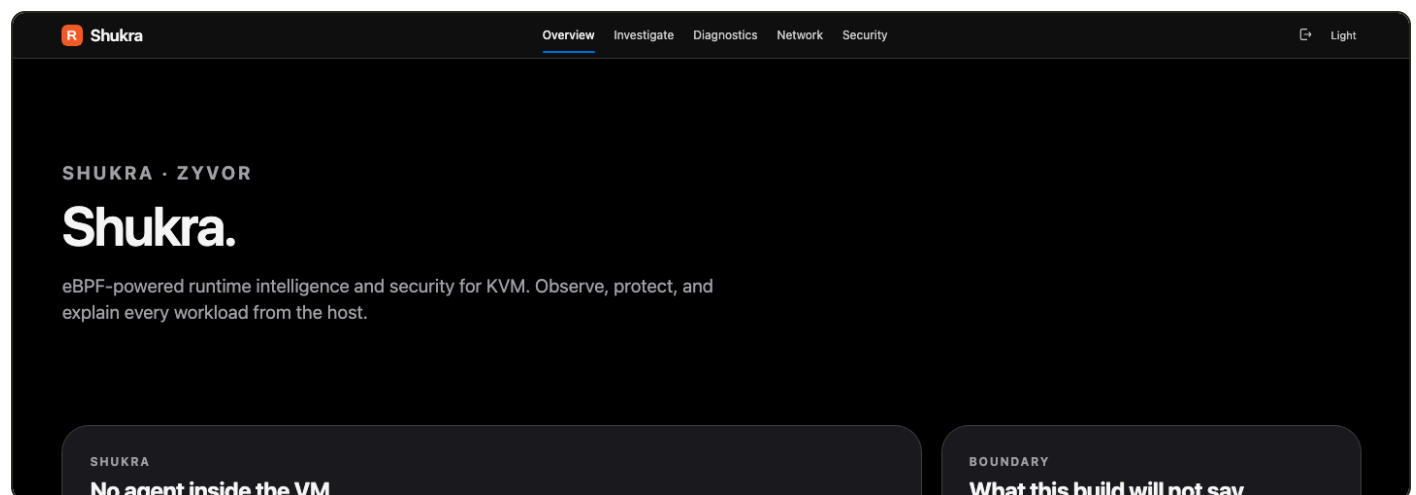
Every answer lists its blind spots. A program that is not measuring says so.

RECENTLY ADDED

- Right-sizing advice
- Learned baselines
- Explain a past time
- Incident bundles
- Noisy-neighbour map
- Who took the vCPU's CPU
- Kernel drops on the VM's tap
- TCP handshake outcomes
- DNS names the guest looks up
- FluxVM guests
- A share of events for each kind
- Filterable connection tables
- Tested on real KVM guests

Standalone eBPF for KVM hosts · QEMU, libvirt, kubevirt and FluxVM · x86_64 and arm64 packages

Open source under the Apache License 2.0. Copyright 2026 Zyvor.



Illustrative: the Shukra overview, rendered from the console's fixture data. UI captures in this brochure are fixtures, not customer data; diagrams are conceptual.

When teams reach for Shukra

A guest can tell you it is slow. Only the host knows why: whether a neighbour took its CPU, the disk stalled, the host dropped its packets or the guest stopped reading its NIC. Shukra sits on the hypervisor and answers that, without installing anything in the VM.

Problems Shukra addresses

- 1 Guest metrics cannot see the host, so "is it us, the disk or a noisy neighbour?" is a guess.
- 2 Slowness is reported after the fact, and the counters that would explain it have already moved on.
- 3 A compromised VM has to be cut off now, without logging in to it and without cutting off the people managing it.
- 4 Agents inside guests are a fleet to install, patch and trust. Someone else's VM is not yours to change.
- 5 VMs are sized once and never revisited: some idle at eight vCPUs while others wait for a CPU.
- 6 The first unusual connection from a VM has no rule yet, because nobody knew to write one.
- 7 Every tool should say what it cannot see. A missing measurement must never look like a healthy zero.

What Shukra is, in one sentence

Shukra is eBPF runtime intelligence and security for KVM: a daemon on the hypervisor that attaches kernel traces, joins them to the QEMU process, and gives an operator a console and a CLI that say what they know, how they know it, and what they cannot see.

Two programs you run

shukrad (on the hypervisor)

Privileged. Loads the eBPF programs, scans `/proc` for QEMU and FluxVM guests, keeps the flight recorder and detection rules, and serves the API and the console on port 30970.

shukractl (anywhere)

An operator CLI that talks to the API only and never loads BPF. Status, doctor, traces, explain, advise, baseline, incident, watch, isolate: one screen to trust first, then the detail.

At a glance

6

eBPF programs, each loaded on its own so one missing tracepoint detaches only that program

16

kernel attach points across kvm, sched, block, net, tap and drops

0

agents, modules or changes inside any guest

14

console pages in five groups: overview, investigate, diagnostics, network and security

28

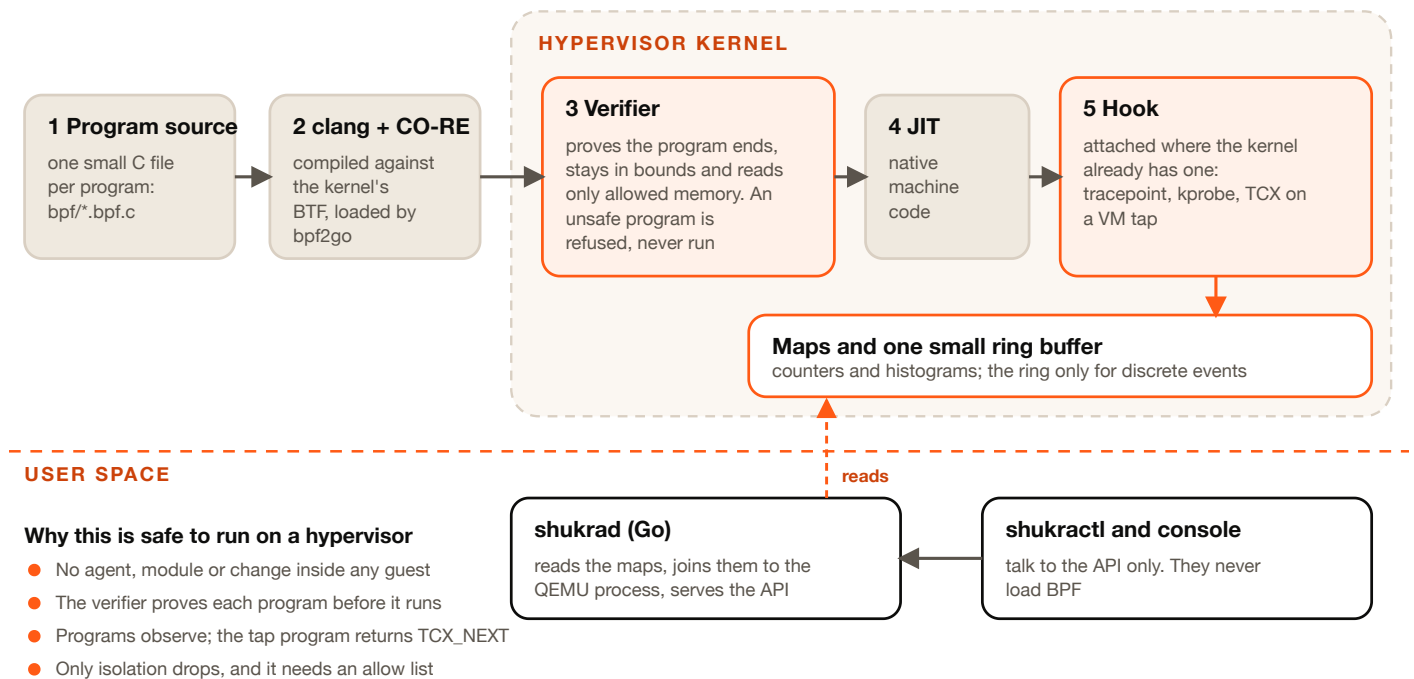
HTTP routes: three for operations, 25 under `/api/v1`, only three of which change state

17

shukractl commands, including `explain`, `advise`, `baseline`, `isolate` and `doctor`

How eBPF does it

eBPF lets Shukra run small, verified programs inside the Linux kernel at the points where VM exits, scheduling, disk requests and packets already pass. That is how it sees a guest from outside: no kernel module, no agent, no change to the VM.



From source to a kernel hook and back to the daemon. Illustrative.

What Shukra attaches to

Tracepoints for KVM exits, the scheduler, block requests and dropped packets; kprobes on TCP connect; and TCX on each VM's tap, on Linux 6.6 and newer.

How the data comes out

Hot paths stay in maps as counters and histograms. A small ring buffer carries only discrete events, and every source of events is sampled or rate-limited.

What keeps it safe

The verifier refuses anything unsafe, and each program is its own object. A kernel that cannot run one reports it detached, with the reason, and the others keep running.

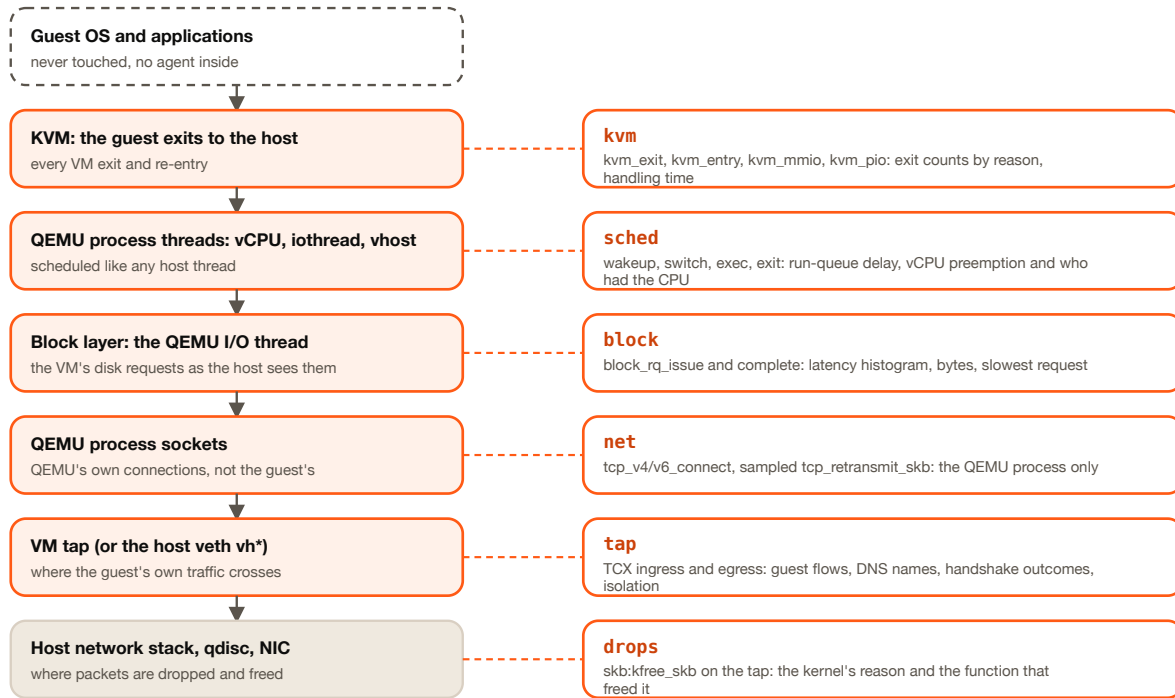
Built once, run on the kernel in front of it. The programs are compiled with CO-RE against kernel type information (BTF) and loaded by `bpf2go`, so one object adapts to the running kernel. Without BTF, clang or root, Shukra still discovers VMs, reports every program as detached with the reason, and never invents a counter.

Why eBPF, not the alternatives

	Agent inside each guest	Kernel module	Shukra with eBPF
Sees host-side causes	no: it only sees the guest's view	yes	yes: exits, run-queue delay, preemption, block, drops
Changes to guests	one install per VM, kept patched	none	none
Risk to the host	lives in someone else's VM	a bug can crash the hypervisor	the verifier proves each program safe before it runs

Where Shukra sits on the hypervisor

A VM's life crosses the same layers on every host. Shukra places one program at each layer that matters and reads names and threads from `/proc`, so the picture of a slow or silent VM runs from the guest's exits to the last function that freed its packet.



Why the KVM and scheduler hooks

A VM is slow for host-side reasons first: it exits too often, or its vCPU threads are runnable and waiting for a CPU. Both are visible only from the host.

Why the tap

The guest's own traffic crosses the host side of its tap. Seeing it there gives the guest's addresses and DNS names, with no agent and without reading application data.

Why the drop tracepoint

Most packet loss is decided inside the host kernel. The tracepoint says why, and which function freed the packet, so a firewall rule and a full queue are told apart.

What it costs (measured)

Program	Measured cost	Where, and the caveat
drops	about 255 ns per drop; 69 a second is 0.002% of one CPU	12-CPU Xeon hypervisor. It runs once per packet dropped anywhere on the host, so scale by your drop rate: a million a second is about a quarter of a CPU
sched	about 490 ns per switch with no VM watched, about 615 ns with ten; about 435 ns before preemption tracking	same host, ten VMs, about 60,000 switches a second: about 11 ms of CPU per second more, under 0.1% of the host. Includes the kernel's own run-time accounting, switched on for the measurement

Measured on a lab hypervisor; results depend on your kernel, hardware and traffic. Costs of the other programs are not published.

The six eBPF programs

Each program is a separate object with its own hooks and its own switch. A host that cannot run one loses only that program, and says why. Hot paths stay in maps; every source of events is sampled or rate-limited.

Program	Hooks	What it records	Needs
kvm	<code>kvm_exit</code> , <code>kvm_entry</code> , <code>kvm_mmio</code> , <code>kvm_pio</code>	exit counts by reason, exit handling time as a histogram (halts excluded) and time per reason	BTF, Linux 5.8+
sched	wakeup, switch, exec, exit	on-CPU time, run-queue delay per thread, vCPU preemption (runnable but off a host CPU, and who had it), exec and exit of QEMU children	BTF, Linux 5.8+
block	<code>block_rq_issue</code> , <code>block_rq_complete</code>	latency histogram, requests, bytes and the slowest request, per direction	BTF, Linux 5.8+
net	<code>tcp_v4_connect</code> , <code>tcp_v6_connect</code> , sampled <code>tcp_retransmit_skb</code>	exact connect counts (IPv4 and IPv6) and 1-in-64 retransmit samples: the QEMU process's sockets	BTF, Linux 5.8+
tap	TCX ingress and egress on each VM tap	the guest's own traffic: per-tap counters, one event per TCP connect, new UDP flow, connection made to the guest, the name in each plain DNS query, what became of each handshake, and isolation	Linux 6.6+, <code>CAP_NET_ADMIN</code>
drops	<code>skb:kfree_skb</code> on each VM tap	what the kernel dropped on the tap, by its own reason and the function that freed it, with Shukra's own isolation subtracted	Linux 5.17+

Never in the way of other tools

The tap program returns `TCX_NEXT`, never `TCX_PASS`, so any other program on the same tap (Cilium, FluxVM's egress program, a `tc` filter) still runs after it. Its links and maps are pinned under `/sys/fs/bpf/shukra/tap`.

Kernel baselines

Any program needs BTF at `/sys/kernel/btf/vmlinux` and `CAP_BPF` (Linux 5.8+). Drop reasons need 5.17, TCX and isolation need 6.6. On arm64, `kvm_pio` does not exist, so `kvm` attaches 3 of its 4 hooks.

Identity comes from the host

Shukra names a VM from the QEMU command line (`-name` or `guest=`, `-uuid`, `ifname=`) and, for libvirt taps passed as file descriptors, the process's `fdinfo`. A FluxVM guest (QEMU, Cloud Hypervisor, Firecracker or `fluxvm-hypervisor`) is named from FluxVM's own record and traced on its host veth. A PID that is not a VMM rolls up to `_host` as `unattributed`. It is never given a made-up VM name.

What a host reports when a program cannot run

Host	Program	Reported
hv-1	tap	attached, one tap per running VM
hv-2	tap	detached, with the reason: the kernel is older than 6.6

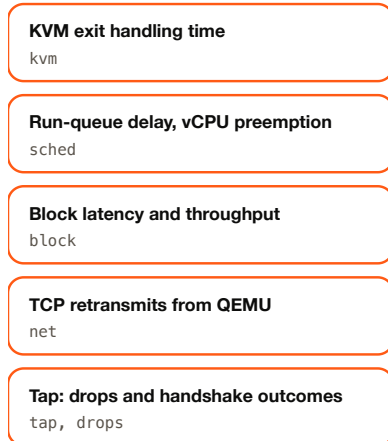
Illustrative. `shukractl programs` shows this per host, and a VM with no measurement has no series at all: "not measured" is never rendered as zero. Percentiles come from log2 buckets and can read up to 2x high.

SLOW-VM SCENARIO · PART 1 OF 4

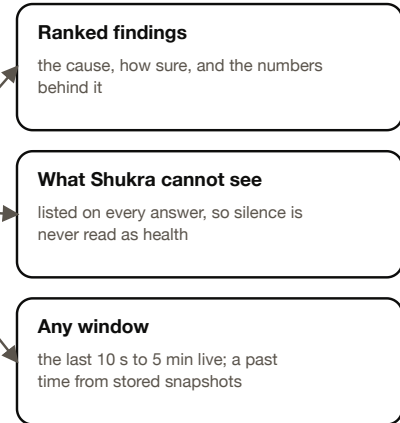
A VM turns slow

It is 03:12. A user says **payment-prod-03** is slow. Nothing inside the guest looks wrong, and the host graphs are flat. The counters that would explain it are already in Shukra's maps, so the operator asks.

EVIDENCE THE HOST ALREADY HAS



WHAT THE OPERATOR GETS



- 1 **The programs count, always.** Exits, preemption, disk requests, connects and tap drops are recorded before anyone asks.
- 2 **shukrad joins them to the VM.** It rescans `/proc` every two seconds, so counters land on the right process and thread: a vCPU, an iothread or a vhost worker.
- 3 **The operator runs `shukractl explain payment-prod-03`** (or opens the Explain page). It weighs the last minute; `--window` takes 10 s to 5 min.

Nine host-side causes, best supported first

Cause	What it points at
host_cpu_contention	the VM's threads wait for a host CPU after being woken: host load, pinning, neighbours
cpu_preempted	the host took CPU away from the vCPUs while they wanted to run
noisy_neighbour	one other VM accounts for at least half of that preempted time
storage_latency	block requests from QEMU take long: the backing device, queue depth, other writers
kvm_exit_handling	the host is slow to service KVM exits, often device emulation
tcp_retransmits	the QEMU process retransmits: host traffic such as migration or a remote disk, not the guest's
guest_traffic_dropped	the kernel drops this VM's packets and Shukra's isolation is not the cause: another program on the tap
guest_not_reading_nic	the tap's queue is full: the guest is stalled, has no working NIC driver or is overloaded
guest_connects_failing	most outbound connections fail: refused means nothing listens, never answered means something drops the SYN

A finding says where to look, not what is wrong inside the guest. Each carries a cause, a confidence (high, medium or low), a summary and the evidence behind it. When nothing crossed a threshold the answer is `no_host_cause` : look inside the guest, which Shukra does not measure.

SLOW-VM SCENARIO · PART 2 OF 4

Who took my vCPU's CPU?

The finding is `cpu_preempted`: the host took CPU away from the VM's vCPUs. On a busy hypervisor the next question is who. Shukra answers it from the scheduler's own switch events, without touching either guest.

payment-prod-03: vCPU threads runnable but off a host CPU, 240 ms in the window

vm:batch-etl-01 190 ms 79%

kworker 50 ms

Who had the CPU

the thread that took it when a vCPU was switched out while still runnable: a VM by name, or a host command

One VM accounts for half or more

then Explain names it: cause noisy_neighbour. `cpu_preempted` stays beside it

What it was doing meanwhile

the culprit's own CPU, block and exit numbers over the same window

Illustrative numbers. The percentages are shares of the VM's preempted time.

How it is measured

`sched_switch` fires when a thread leaves a CPU. If a QEMU thread leaves while still **runnable**, it was taken off while it wanted to run: the program remembers who took the CPU and, when the thread next runs, adds the wait to the pair (this thread, that thread). A thread that goes to sleep is dequeued first, so sleeping is never counted.

What the answer names

- Another VM, as `vm:<name>`, or the VM's own other threads.
- Or a host command with its instance suffix removed: `kworker/3:1` and `kworker/u16:2` are both `kworker`.
- For each culprit VM, what it was doing over the same window: its on-CPU time and KVM exits.

What it is not

- Not the steal counter a guest reads. It is the host's view of the guest losing its processor.
- vCPU threads only. An iothread that waited for a CPU is not the guest losing its processor.

```
$ shukractl trace contention --window 5m
vm=osboxes-debian preempted=1882612ns over 12 preemptions (other VMs 0ns, its own threads 0ns, host tasks 1882612ns)
taken by host tasks: cilium-agent 1031780ns, kubectrl 321523ns, hubble-relay 188161ns, containerd-shim 103092ns
as printed in the after-the-fact tutorial: here the culprit was the host's own tasks, not another VM
```

CONTENTION

Whose CPU another VM took (last 1m0s)

VICTIM	TAKEN BY VM	TIME	SHARE OF ITS PREEMPTION
payment-prod-03	api-01	190 ms	79%
redis-01	api-01	60 ms	63%
api-01	payment-prod-03	25 ms	36%

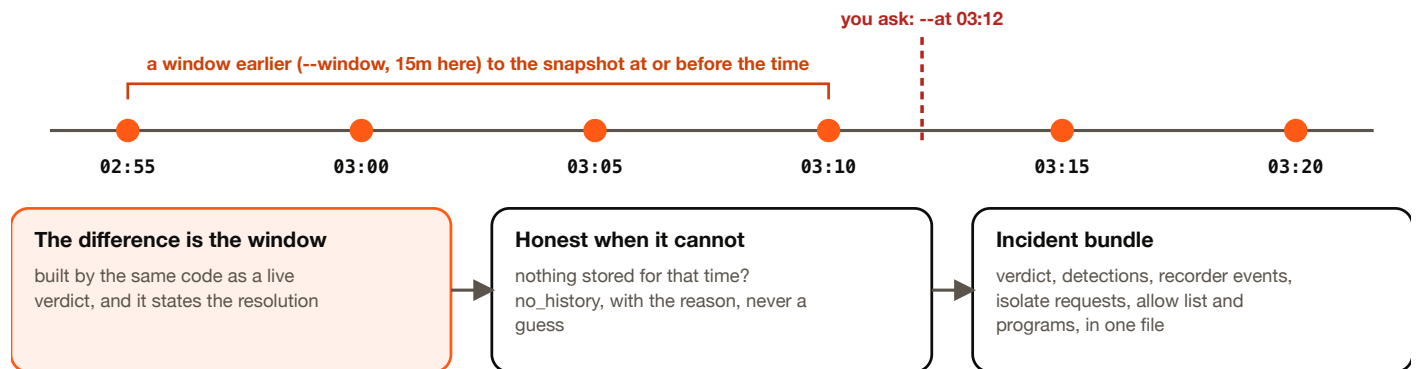
Illustrative: the Contention page, rendered from fixture data.

SLOW-VM SCENARIO · PART 3 OF 4

Why was it slow at 03:12?

It is half past five. The last minute, which Explain weighs by default, is no help. With a data directory, the daemon has been storing a coarse snapshot every five minutes, so the operator can ask about a past time and say how far to trust the answer.

A COARSE SNAPSHOT EVERY 5 MINUTES (with `-data-dir`)



- 1 Ask for the verdict at that time.** `shukractl explain payment-prod-03 --at -2h30m`, or an RFC 3339 time. The window is 5 minutes to 6 hours, 15 minutes by default.
- 2 Read the resolution line first.** Snapshots are five minutes apart, so this is an average around the time, not the second it happened. A short burst can be diluted into nothing, and the answer says so.
- 3 Find who was involved** with `shukractl trace contention` if the finding is `cpu_preempted` or `noisy_neighbour`.
- 4 Attach one file to the ticket:** `shukractl incident payment-prod-03 --at -2h30m --out bundle.json`, written with mode 0600.

```
$ shukractl explain osboxes-mint --at -2h30m
EXPLAIN why was this VM slow? (window: 5m0s) (at: 2026-09-20T04:50:27Z)
  resolution: 5m0s snapshots: the verdict stands on 2026-09-20T04:40:53Z and 2026-09-20T04:45:53Z
  findings (best supported first)
    [low] no_host_cause: Nothing host-side crossed a threshold. If the guest is slow, the cause may be inside it, which
    this build does not measure.
as printed in the after-the-fact tutorial. A calm host is a real answer: it says the host was fine.
```

What the incident bundle holds

The verdict; the window's detections; what the flight recorder saw of that VM (at most 500 events); isolate and release requests; whether isolation is available and the management networks an isolated VM keeps; and which programs were attached. Nothing else from the daemon's configuration: no key, no listen address.

What it cannot tell you

The second it happened. Anything before the daemon first stored a snapshot, or while it was not running. Anything inside the guest. And with nothing stored, it says `no_history` and why, instead of guessing. Retention is set by size: about a day or two at ten VMs.

The bundle is as sensitive as the event list. It names your VMs, the addresses they connected to and the DNS names they looked up. Treat it like the event list.

SLOW-VM SCENARIO · PART 4 OF 4

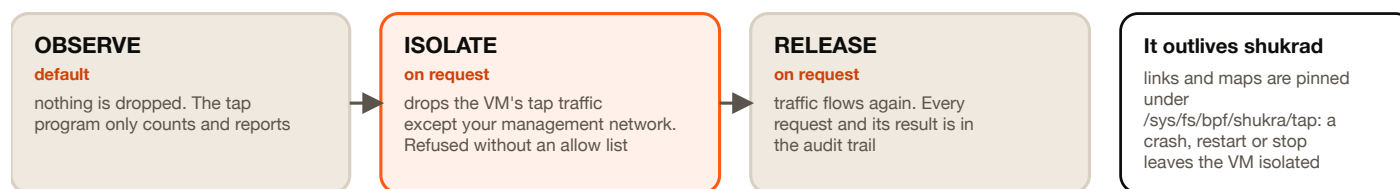
Lost traffic, then contain

Another ticket: **api-01** cannot reach a service. Is the destination refusing, is something dropping the packets, is the guest not reading its NIC, or is Shukra itself in the way? The tap answers each connection, and the kernel names each drop.

EVERY TCP CONNECT THE GUEST MAKES ENDS AS ONE OF FOUR



CUT A VM OFF WITHOUT TOUCHING IT



What the kernel dropped, and why

Reason on the tap	What it means	Where to look
TC_INGRESS, TC_EGRESS	a tc or TCX program on the tap dropped a frame; if Shukra accounts for it, that was isolation	<code>bpftool net show dev <tap></code> : another program (Cilium, a tc filter)
FULL_RING	the tap's queue is full: the guest is stalled, has no working NIC driver, or is overloaded	the VM itself
NETFILTER_DROP	iptables or nftables	the rule whose packet counter is moving
QDISC_DROP, CPU_BACKLOG	the host's own queues are overflowing	host load and qdisc settings

Each drop is also attributed to the function that freed the packet. Shukra's own isolation drops are subtracted, so "is it us?" has an answer: `shukractl trace drops`. Reason names are read from the running kernel's BTF and never hard-coded; one it cannot name is shown as `reason N`, not guessed.

- 1 Read the outcome.** `shukractl trace tap --vm api-01` shows accepted, refused, never answered and blocked per direction, with the guest's handshake time.
- 2 Read the drops.** `shukractl trace drops --vm api-01` says what the host kernel dropped on the tap, by reason and function, and whether it was Shukra.
- 3 If the VM must be cut off,** `shukractl isolate api-01` drops its tap traffic except your management network. It prints whether the kernel took the change.

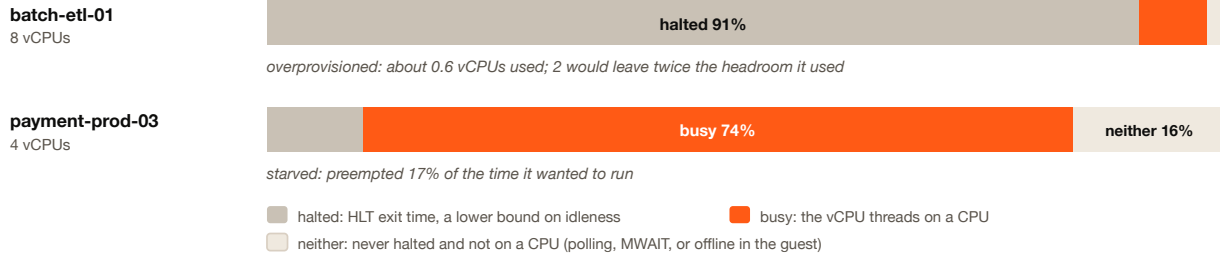
Isolation is real, and conditional. It needs an explicit allow list (`-isolate-allow`) and Linux 6.6 or newer, and is refused without them, because a VM that could reach nothing could not be reached by the host that isolated it. `applied` is true only after the kernel took the change on every tap; a half-applied isolation is reported `partial`, not silently reopened. It blocks the VM's tap: it does not touch vhost-user or SR-IOV interfaces.

BEYOND THE INCIDENT · CAPACITY

Is this VM the right size?

A VM that idles at eight vCPUs wastes host CPU, and one that wants CPU and is not getting it costs its users. The counters Shukra already has answer both, in both directions. The answer is advice for a person, with the numbers and how sure it is. Nothing here resizes a VM.

WHAT THE ADVICE STANDS ON: THE SHARE OF EACH VM'S vCPU CAPACITY OVER THE WINDOW



Overprovisioned

2 or more vCPUs, halted at least 80% and busy at most 20%

Starved

busy at least 50%, and preempted 10% or a vCPU waiting 2 ms for a CPU

Nearly idle

one vCPU, halted at least 95%: a consolidation candidate

Illustrative fixture numbers, over a five-minute window.

What it prints

`shukractl advise` shows, per VM, the share of the window its vCPUs were halted, busy and preempted, then advice: `overprovisioned` (with a size that leaves twice the headroom it used), `starved`, `nearly_idle`, `no_change`, `not_enough_data` or `idle_unavailable`.

A lower bound, and a gap it shows

Idleness is HLT exit time, so a guest that idles by polling reads as busy. The share that is neither halted nor on a CPU is reported, and when it is 30% or more, `no_change` says that is not proof the VM is right-sized.

Only where the CPU names the exit

Exit reasons are named on Intel hosts. Elsewhere Shukra says `idle_unavailable` and claims nothing about over-provisioning; starvation is still reported.

How sure

Every piece of advice carries a confidence and its evidence. A window under three minutes lowers it; under 20 seconds of history the answer is `not_enough_data`. Only vCPU threads count.

RIGHT-SIZE							
Is each VM the right size? (last 5m0s)							
VM	VCPUS	HALTED	BUSY	PREEMPTED	NEITHER	WORST RUN-QUEUE WAIT	ADVICE
batch-etl-01	8	91%	7% (0.6 vCPUs)	0%	2%	65.5 µs	overprovisioned MEDIUM This VM has 8 vCPUs and used about 0.6 of them. 2 would leave it twice the headroom it used. Halted 91% of the time.
payment-prod-03	4	10%	74% (3.0 vCPUs)	17%	16%	33 ms	starved HIGH This VM wants more CPU than the host gives it. Preempted 17% of the time it wanted to run.

Illustrative: the Right-size page, rendered from fixture data. It sits in the Diagnostics group.

Advice, not an action. Starved VMs are told to reduce what they compete with (pinning, moving a neighbour) before adding vCPUs, because more vCPUs mean more threads competing for the same host CPUs. A person decides.

BEYOND THE INCIDENT · WHAT IS NEW

What is new for this VM?

A watchlist needs someone to know what is bad. A baseline needs no one to know: it learns what each VM normally does and reports the first time it does something new. A web server that has talked to the same three networks for a month and suddenly talks to a fourth is worth a look, and no rule had to name that network in advance.

A VM IS ONLY OBSERVED FIRST; THEN THE FIRST SIGHTING OF ANYTHING NEW IS REPORTED, ONCE

learning: recorded, nothing reported

first time the VM is seen

reporting: a first sighting becomes one detection

24 h later (learn: 24h is the default)

new-destination

severity medium

a network the guest never contacted before: the /24 of an IPv4 address, the /64 of an IPv6 address

new-dns-suffix

severity low

a site it never looked up: example.co.uk, not the host name in front of it

new-inbound-peer

severity medium

a network that never connected in to the guest before

Bounded against a guest that controls what it says

Fixed-size sets (2048 per kind); at most 20 alerts per VM per day, then one baseline-cap detection; unseen items are forgotten after 30 days. Off unless the rules file has a `baselines:` section.

Coarse on purpose

A whole network and a whole site, not an address and a host, so a service that moves between neighbouring addresses or a site with a new subdomain is not new each time. A short built-in list of 63 two-label suffixes (`co.uk`, `com.au`) keeps names right without the full public suffix list; for one it lacks, a name is learned too wide, which is quieter and never louder.

Only what the tap sees

Everything is learned from the VM's own tap, so every detection is guest-attributed. A host connect belongs to QEMU and is never learned. A VM first seen later has its own learning period, so a new VM does not alarm for its first day.

```
$ shukractl baseline
```

```
BASELINE learning period 24h0m0s, at most 20 new-item alerts per VM per day
```

```
vm=web-01 learning until 2026-09-21T03:00:00Z (destination 4, dns-suffix 2, inbound-peer 0) new alerts today 0, held back in all 0
```

```
vm=db-01 reporting (destination 9, dns-suffix 1, inbound-peer 1) new alerts today 3, held back in all 5
```

```
[medium] new-destination web-01 contacted 192.0.2.0/24 for the first time (tcp to 192.0.2.44:22)
```

```
as printed in docs/baselines.md (documentation example addresses)
```

Turned on by you

Off unless the rules file has a `baselines:` section. It needs `-data-dir:` without it, learning starts over with every restart and a daemon that restarts more often than the learning period would never report anything.

`shukractl doctor` warns about exactly that.

Forgetting is recorded

`shukractl baseline VM --forget` (admin key only) restarts a VM's learning after a deliberate change. It could also hide a change, so it is recorded as a `baseline-forgotten` detection naming who did it.

A guest can hide inside its own learning period. Whatever it does in the first 24 hours becomes normal. Start a VM you do not trust under a watchlist (`destinations`, `dns`), not a baseline. Past the daily cap, new items are still learned and counted but not reported, and one `baseline-cap` detection says so.

What Shukra reads, and what it never does

The accurate promise is stronger than "no agent": nothing runs inside a guest, no application data is read, and the one thing read beyond headers is stated here, can be turned off, and is bounded.

By default

Metadata only. Counters and histograms from the host: exits, scheduling, block requests, TCP connects, tap packet counts. From a tap: headers, so flow addresses and ports, handshake outcomes and kernel drop reasons. VM names from the QEMU command line.

One bounded exception

DNS names. For a plain DNS query on UDP port 53, the first question (at most 128 bytes: the name and the type). `-dns-events=false` turns it off. Answers, DNS over TCP, TLS and HTTPS are not read.

Never

Anything inside a guest. Application payloads. The guest's own steal counter. Which process in the guest made a connection. Keys in any API response, in `doctor` or in `systemctl show`.

Question	Answer
Does anything run inside the VM?	No. There is no agent. Everything is read from the host: <code>/proc</code> , kernel tracepoints, and the host side of each VM's tap.
Are host TCP events the guest's?	No. Host connects are the QEMU process's own sockets, labelled <code>qemu-process</code> and <code>guest_attributed: false</code> . Only events seen on a VM's tap are <code>guest-tap</code> and <code>guest-attributed</code> , and only for a tap that belongs to a VM in the current scan. A tap no VM owns stays <code>unattributed</code> .
Can a guest change what is attributed to it?	No. Attribution comes from the tap's owner, found on the host, not from anything in a packet.
Can a hostile guest hurt the daemon?	It can generate load, not growth. Tables keyed by packet contents are fixed-size LRUs; events from a tap are rate-limited to 200 per second; DNS names are cut at 253 bytes with every non-printable byte replaced, so a name cannot carry an escape sequence into a log.
Can a guest game the baselines?	Only its own learning period: whatever it does in the first 24 hours becomes normal. Its sets are fixed-size, alerts are capped per VM per day with one <code>baseline-cap</code> detection, and a forget request is recorded with who made it. Untrusted VMs belong under a watchlist.
Does the advisor change anything?	No. Right-size advice is text for a person, with the evidence and a confidence. Nothing resizes a VM.
Who can isolate a VM?	The admin key only, which is also the only key that can forget a learned baseline. A separate read-only key reads everything, including <code>/metrics</code> and the stream, and cannot change anything. Give that one to Prometheus.
What does an incident bundle expose?	VM names, addresses and DNS names, as the event list does. It holds nothing from the daemon's configuration except the allow list. It is written with mode 0600.
What does the daemon need to run?	Root with six capabilities and a read-only filesystem apart from its data directory (<code>/0700</code>). <code>CAP_SYS_PTRACE</code> and <code>CAP_DAC_READ_SEARCH</code> are used only to read a libvirt VM's tap names from <code>/proc/<pid>/fd</code> ; drop them if you run no libvirt VMs.

What this release will not pretend. It can say *which VM and which address*, not *which process inside the guest*. A VM with user-mode networking has no tap and so no guest traffic, and `shukractl doctor` names it. The scheduler program records the command name of a host task that took a vCPU's CPU, so names like `kubectl` can appear in the API. Treat the data directory as sensitive, and alert sinks as a place that data leaves the host.

The console: fourteen pages, five groups

The console is served by the daemon on port 30970 and talks only to its API. Every page says what it knows, how it knows it, and what it cannot see.

Overview

Programs attached, VMs found, detections, and the host banner.

Investigate

Virtual machines, Flight recorder, Explain.

Diagnostics

KVM, Scheduler, Contention, Right-size, Block, Programs.

Network + Security

Host connections, Drops. Detections, Isolate.

VM: payment-prod-03 | At: dd/mm/yyyy, --:-- -- | Look back: last minute | Download incident bundle

FINDINGS

Where to look, best supported first

Looking at: the last 1m0s

Noisy Neighbour HIGH CONFIDENCE

Another VM, api-01, took most of the CPU this VM's vCPUs were denied. Look at api-01's load, and at CPU pinning or separating the two onto different cores.

api-01 took 190 ms of the 240 ms the vCPUs were preempted (79%). shukractl trace contention shows what api-01 was doing meanwhile.

Storage Latency HIGH CONFIDENCE

Block requests from QEMU take long. Look at the backing device, its queue depth and other writers.

Block write p99 is up to 33.6 ms (slowest 28 ms) across 940 requests.

Cpu Preempted HIGH CONFIDENCE

The host took CPU away from this VM's vCPUs. Look at what else is running on those host CPUs: other VMs, host

Explain: ranked findings in the daemon's own words, an At field for a past time, and the incident bundle download.

TRACE

Packets the kernel dropped on VM taps

VM	TAP	KERNEL DROPS	SHUKRA DROPPED	OTHER DROPS	GUEST NOT READING
payment-prod-03	tap7	1290	0	1204	0
api-01	tap1	14	0	3	11
redis-01	tap2	0	0	0	0

Drops: what the kernel dropped on each tap, by reason and freeing function, and whether it was Shukra.

ENFORCEMENT

Isolate is enforced on the VM tap

An isolated VM keeps ARP, IPv6 neighbour discovery and traffic to or from 10.0.0.0/24, 10.0.9.4. Everything else on its tap is dropped.

VM: payment-prod-03 | Isolate... | Release

Cut payment-prod-03 off from the network? Its tap will drop everything except ARP, IPv6 neighbour discovery and 10.0.0.0/24, 10.0.9.4. It stays isolated until you release it. It stays isolated if the daemon stops or crashes. Release it here, or with shukrad -detach-all if the daemon is down.

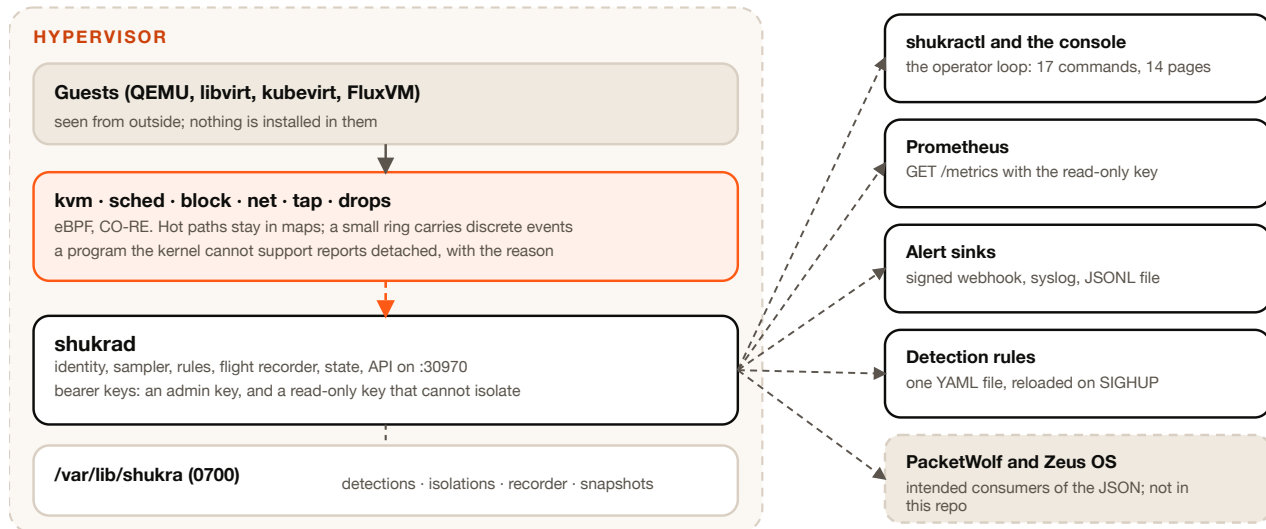
Isolate payment-prod-03 | Cancel

Isolate: cut a VM off except the management network, with a confirmation step and what survives a daemon crash.

Illustrative: all captures are rendered from the console's fixture data. They are not a live host or customer data.

Rules, alerts, state and the API

Detection is one YAML file. Alerts leave through sinks you choose. State survives a restart when you give the daemon a directory. Everything an operator sees in the console is a plain API call.



Detection rules

One file (`--watchlist`), re-read on `systemctl reload shukra` and checked offline with `shukractl rules check`. A bad file keeps the previous rules.

```

destinations: # out to, or in from
  - {cidr: 185.0.0.0/8, name: egress, severity: high}
ports:
  - {port: 22, name: ssh-in, dir: in}
dns:
  # a name the guest looked up
  - {name: pool, suffix: nanopool.org}
thresholds: # a per-VM metric over a window
  - {name: slow-disk, metric: block_write_p99_ms, value: 50}
  
```

A detection keeps the attribution of the event that caused it. A rule says nothing until a full window of history exists. Or add a `baselines:` section and let each VM teach it what is normal.

Alert sinks

- **Signed webhook:** HMAC-signed when a secret is set.
- **Syslog** and a **JSONL file**.

State that survives a restart

File	Written
<code>detections.jsonl</code>	on every detection
<code>isolations.jsonl</code>	on every isolate request
<code>recorder.json</code>	every minute and at clean shutdown
<code>snapshots.jsonl</code>	every 5 minutes; read on demand
<code>baselines.json</code>	every minute, if it changed

Directory `0700`, files `0600`, each log rolls at 16 MiB.

Monitor the daemon

Endpoint	Auth	What
<code>/healthz</code> , <code>/readyz</code>	none	process is up; <code>200</code> after the first scan, <code>503</code> before
<code>/metrics</code>	bearer	Prometheus text. A VM with no measured counters has no series, not a zero
<code>/api/v1/advice</code> , <code>/baseline</code>	bearer	right-size advice per VM; what each VM has learned as normal
<code>/api/v1/stream</code>	bearer	server-sent events; resume with <code>Last-Event-ID</code> . Every event carries a <code>seq</code> that only grows
<code>POST /api/v1/isolate</code> , <code>/release</code>	admin key	with <code>POST /api/v1/baseline/forget</code> , the only three routes that change state

Deploy, requirements, and how it is tested

Shukra is a systemd unit on the hypervisor, not a Helm chart, because eBPF has to run where the VMs run. One script installs it; a tarball and a `.deb` cover hosts without a compiler.

Install

```
$ ./scripts/deploy-remote.sh 10.0.1.5 sus
# build on the host, install, start
# shukra.service on :30970, run doctor
$ make dist
# tarball and .deb (amd64, arm64)
```

Set a real key on any host that is not a lab. Without `SHUKRA_API_KEY` the daemon uses a dev token and says so, and `doctor` reports it as a failure on a non-loopback address.

Requirements

To get	You need
daemon, CLI, API	Go 1.25+ to build, or a release package
kvm, sched, block, net	kernel BTF and <code>CAP_BPF</code> (5.8+)
drops	Linux 5.17+
tap and isolation	Linux 6.6+, <code>CAP_NET_ADMIN</code>
console	Node 22 to build

Tested for real, not just unit-tested

EVERY PUSH AND PULL REQUEST

Go tests (366 test functions), the console's 45 tests and its build, the programs loaded into the runner's kernel, the installer.

REAL KERNEL, TAP RIG

Guest traffic, isolation, drops and every handshake outcome, in a network namespace with a real veth, with exact counts.

REAL KVM GUESTS

Two guests booted by fluxvm on a runner with `/dev/kvm`: guest events are attributed, packet counts equal the kernel's, taps come off when the VMs go.

Exact, not approximate. On GitHub's kernel, 17 outbound attempts came out as 5 accepted, 6 refused, 4 never answered and 2 blocked, with 1 retransmit, and attempts = accepted + refused + never answered + blocked held. Isolation is shown to survive `kill -9`, a restart and a graceful stop, and to be lifted by `-detach-all`.

Release

Pushing a `v*` tag builds the tarball and `.deb` for amd64 and arm64. Shukra has no tagged release yet: the changelog lists what has merged to `main`.

Honest limits. The live-guest test fails, rather than skips, on a runner with no KVM. A SYN is judged "never answered" after 3 seconds, and under a SYN flood the oldest pending entries are forgotten uncounted (attempts still counts every one).

Where Shukra fits, and a buying checklist

Shukra is one Zyor product, open source, and it works alone. What it does not do is stated here so the choice is clear.

What Shukra is

Runtime intelligence and security for KVM from the hypervisor: what a host measures about its VMs, what became of a guest's connections, and a way to cut a VM off. Guests untouched.

What it is not

Not an agent in the guest, so it cannot name a process inside one. Not a network firewall: isolation is one VM's tap, with an allow list. Not a CNI or a network policy engine: it sits on the hypervisor.

FluxVM

Recognised: QEMU, Cloud Hypervisor, Firecracker and `fluxvm-hypervisor` guests are named from FluxVM's own record and traced on their host veth.

PacketWolf and Zeus OS

The intended consumers of Shukra's JSON. They are not in this repository, and nothing here depends on them.

Buying checklist

- 1 Run `shukractl doctor` on a candidate hypervisor: exposure, what is attached, what it cannot see, each finding with a fix.
- 2 Start observe-only. Read Overview, Programs and one real slow VM's Explain. Nothing is dropped.
- 3 Check what the kernel supports: 5.17 for drops, 6.6 for the tap and isolation. Read what each host reports as detached, and why.
- 4 Write a small rules file (add a `baselines:` section if you want first sightings reported), check it offline with `shukractl rules check`, and wire one alert sink. Give Prometheus the read-only key.
- 5 Set a real API key and TLS before any non-lab host is reachable.
- 6 Rehearse isolate on a test VM with an allow list: confirm it is refused without one, survives a daemon restart, and releases.
- 7 Run a day with a data directory, then ask `explain --at` and `incident` about a past time. Read the resolution line.
- 8 Run `shukractl advise` and read the "neither" figure before trusting an over-provisioned verdict; it is advice, and a person decides.
- 9 Decide whether PacketWolf or Zeus OS will consume the JSON, and which key they get.

See Explain Attribute Contain Integrate Trust

every VM from the host · ranked causes with evidence · who took the CPU · a VM cut off without touching it · Prometheus, webhook, syslog, JSONL · tested on real kernels and real KVM guests

See the VM. Explain the slowdown. Touch nothing inside the guest.

Talk to Zyor → zyvor.dev · source: github.com/zyvorai/shukra

© 2026 Zyor · Apache License 2.0 · eBPF runtime intelligence and security for KVM. Screenshots are rendered from the console's fixture data; diagrams are conceptual; measured figures are quoted from the project's own documentation and depend on your kernel, hardware and traffic.